Finite and infinite bases for generalized stacks

Columbus Labs
info@columbus-labs.com

14 October 2026

© 2026 Columbus Labs. Licensed under CC BY 4.0.

Abstract

Atkinson's generalized (r, s)-stack inserts each input entry into one of its top r positions and outputs an entry from one of its top s positions. The permutations it can sort form a class closed under taking patterns. Atkinson determined finite bases for this class when $r = 1$ or $s = 1$ and when $r = s = 2$, and Vatter's recent problem list records that it is not known whether these bases are all finite. We show that they are not. For every pair $r, s \geq 2$ other than $r = s = 2$, both the class of sortable permutations and the class of generated permutations have infinite bases. We give explicit infinite families of basis elements, one family when $r \neq s$ and two when $r = s \geq 3$, and prove them by following literal stack configurations. Consequently one generalized stack has a finite basis exactly when $\min(r, s) = 1$ or $r = s = 2$. We also determine the effect of a physical capacity H that counts every held entry. For the same widths, the capacity-H basis is finite exactly when $H \leq \max(r, s) + 1$, in which case the class is a maximum-drop class with $H!$ basis elements, and it is infinite for every larger H. Since an (r, s)-stack is a $\mathcal{C}$-machine with a finitely based container class and s selectable pop positions, this also answers negatively a question of Vatter on such machines. Lean 4 checks the original-machine infinite-basis statements and the capacity threshold in the form of a finite-side basis-length characterization and unbounded basis lengths above the threshold. Additional certificates prove the count $H!$ and the exact family displacement widths. The cited finite unbounded cases remain outside the certificates.

1 Introduction

1.1 The machine and the question

A *generalized stack* with push width $r \geq 1$ and pop width $s \geq 1$, or (r, s)-stack, was introduced by Atkinson [1]. It is a single stack, which we write top first; the *depth* of a held entry is the number of entries above it. At each step the machine either reads the next input entry and inserts it into slot j of the current stack, where $0 \leq j \leq \min(r - 1, h)$ and h is the current height, or it removes an entry of depth at most $s - 1$ and appends it to the output. There is no bypass: an entry reaches the output only after it has been inserted. The $(1, 1)$-stack is the ordinary stack.

A permutation π of $\{1, \ldots, n\}$ is *sortable* if some history with input $\pi_1, \ldots, \pi_n$ outputs $1, \ldots, n$, and *generated* if some history with input $1, \ldots, n$ outputs π. Write $\mathcal{S}(r, s)$ and $\mathcal{G}(r, s)$ for these classes. Relabelling each entry by its input position shows that $\mathcal{G}(r, s)$ consists of the inverses of the members of $\mathcal{S}(r, s)$ (Lemma 2.2). Both classes are closed under taking patterns (Lemma 2.1), so each is described by its *basis*, the set of minimal permutations not in it. Since inversion preserves

pattern containment, the two bases are inverse to each other, and in particular they are finite or infinite together.

Knuth's characterization of the ordinary stack gives the basis $\{231\}$ for $\mathcal{S}(1,1)$ [3]. Atkinson [1] showed that the bases are finite whenever $r = 1$ or $s = 1$, and found the eight basis elements of $\mathcal{S}(2,2)$. Vatter's problem list [2, Problem 5.10] asks for the bases for general $r, s \geq 2$ and records that it is not even known whether they are all finite.

1.2 Results

Theorem 1 (infinite bases). For all integers $r, s \geq 2$ with $(r,s) \neq (2,2)$, the classes $\mathcal{S}(r,s)$ and $\mathcal{G}(r,s)$ have infinite bases.

Together with Atkinson's finite bases this gives the classification.

Corollary 2. For $r, s \geq 1$, the class of permutations sortable by one (r,s)-stack, equivalently the class it generates, is finitely based if and only if $\min(r,s) = 1$ or $r = s = 2$.

We also consider a *physical capacity* $H \geq 0$: a history has capacity H if the stack never holds more than H entries, where an entry that is inserted and immediately removed still counts while it is held. Let $\mathcal{S}_H(r,s)$ and $\mathcal{G}_H(r,s)$ be the permutations sortable, respectively generated, by histories of capacity H. These classes are again closed under patterns (Lemma 2.1). For a permutation π of length n let

$$D(\pi) = \max_{1 \leq i \leq n} (i - \pi_i)$$

be its maximum drop, with $D = 0$ for the empty permutation.

Theorem 3 (sharp capacity threshold). Let $r, s \geq 1$ and $H \geq 0$.

(a) If $H \leq \max(r,s)+1$, then a nonempty permutation π lies in $\mathcal{S}_H(r,s)$ if and only if $D(\pi) \leq H-1$. For $H \geq 1$ the basis of $\mathcal{S}_H(r,s)$ consists of the $H!$ permutations of length $H+1$ that end with 1, and for $H = 0$ it is $\{1\}$. In particular these bases are finite.

(b) If $r, s \geq 2$, $(r,s) \neq (2,2)$ and $H \geq \max(r,s) + 2$, then $\mathcal{S}_H(r,s)$ and $\mathcal{G}_H(r,s)$ have infinite bases.

The threshold $\max(r,s) + 2$ is therefore exact for every pair of widths covered by Theorem 1.

Theorem 1 also bears on the restricted containers of Albert, Homberger, Pantone, Shar and Vatter [6]. A $\mathcal{C}$-machine stores entries in a container whose contents must always be order-isomorphic to a member of a permutation class $\mathcal{C}$, and outputs only the leftmost stored entry. Vatter [2, discussion following Problem 5.13] allows the output of any of the leftmost s entries, calling the result a $(\mathcal{C}, s)$-machine, and asks whether every $(\mathcal{C}, s)$-machine with $\mathcal{C}$ finitely based generates a finitely based class. Let $\mathcal{C}_r = \mathrm{Av}(S_r \oplus 1)$ be the class of permutations in which no entry has r smaller entries before it; it is finitely based.

Corollary 4. For all $r, s \geq 2$ with $(r,s) \neq (2,2)$, the $(\mathcal{C}_r, s)$-machine generates a class with an infinite basis. In particular, for every $s \geq 3$ the $(\mathrm{Av}(123, 213), s)$-machine does. Hence a finitely based $\mathcal{C}$ does not in general give a finitely based class generated by a $(\mathcal{C}, s)$-machine.

The proof (Section 5) shows that the $(\mathcal{C}_r, s)$-machine generates exactly $\mathcal{G}(r,s)$. Vatter [2] observed this identification for $s = 1$ and for the $(2,2)$-stack.

The proof of Theorem 1 is constructive. For $2 \leq r < s$ we give a family $R_{r,s,m}$, $m \geq 0$, of permutations of length $2s + r + 2m$ that are basis elements of $\mathcal{G}(r,s)$ (Theorem 3.1). For $r = s \geq 3$ we give two families $A_r(k)$ and $B_r(k)$, $k \geq 0$, of lengths $3r + 7 + 3k$ and $3r + 8 + 3k$ (Theorem 4.1).

Inversion and a reversal duality transfer these to $\mathcal{S}(r,s)$ and to $r > s$ (Section 2.2). All the families have bounded *width* $w(\pi) = 1 + \max_j(\pi_j - j)$, which gives Theorem 3(b) (Section 5).

The rejection arguments follow one entry at the bottom of the stack. In the unequal case the first r entries read fix a bottom entry that can never be undercut, the low outputs force it to be a particular entry P, and the final record buries it under exactly s entries. When $r = s$ this pinning fails, because an entry at depth $r-2$ can be undercut. Our equal-width families instead hand the pin from P to a *carrier* entry C, which is kept at depth exactly $r-2$ by steps that each read two entries, while a queue of $r-2$ older entries circulates above it. For every single deletion we write down an accepting history. The families were found with the help of computer searches for small widths and then generalized symbolically; the proofs do not depend on how they were found.

1.3 Verification and scope

The Lean 4 kernel [5] checks the infinite-basis statements, the finite-side class and basis characterizations, and the capacity threshold in the basis-length form specified in Section 6. The additional source-only certificates prove the finite cardinality deduction, the count $H!$ and the exact family displacement widths. The cited finite unbounded cases retain their written-proof scope. Finite computations are reported as checks, never as proofs. No historical priority is claimed: the bounded literature search behind this manuscript is described in Section 7.

2 Preliminaries

Histories, inputs and outputs are as in Section 1.1. Labels are always the original values; a deleted value is simply omitted. For a permutation π and a value v, write $\pi \setminus v$ for the word obtained by deleting v, and $\mathrm{st}(\pi \setminus v)$ for its standardization.

2.1 Patterns

Lemma 2.1 (deletion). Let a history of capacity H have input α and output β, and let v be an entry of both. Deleting the insertion and the removal of v gives a history of capacity H with input $\alpha \setminus v$ and output $\beta \setminus v$. Consequently $\mathcal{S}(r,s)$, $\mathcal{G}(r,s)$, $\mathcal{S}_H(r,s)$ and $\mathcal{G}_H(r,s)$ are closed under taking patterns, and a permutation not in one of these classes whose single deletions all lie in it is a basis element.

Proof. While v is held, every other operation acts on a slot at most one larger than it would without v; removing v lowers such a slot by one when v lies above it, and otherwise leaves it unchanged. Slots only decrease, so all width conditions remain true, and every visited height decreases or stays the same. Relabelling by standardization does not affect slots. □

2.2 Inversion and duality

Lemma 2.2 (inversion). $\pi \in \mathcal{S}_H(r,s)$ if and only if $\pi^{-1} \in \mathcal{G}_H(r,s)$, and the same holds without capacity.

Proof. Rename every entry of a sorting history by its position in the input. The operations are unchanged, the input becomes $1, \ldots, n$ and the output becomes π^{-1}. The renaming is reversible. □

Lemma 2.3 (duality). Let π^{rc} be the reverse-complement of π. Then $\pi \in \mathcal{G}_H(r,s)$ if and only if $\pi^{rc} \in \mathcal{S}_H(s,r)$, and the same holds without capacity.

Proof. Read a history backwards. Removals become insertions and insertions become removals at the same slots, so push and pop widths exchange and every visited stack is visited again. The output π becomes the input, read backwards, and the input $1, \dots, n$ becomes the output, read backwards. A removed entry had depth below s, so the new insertion uses a slot below the new push width s, and symmetrically. Complementing all labels turns this into a history of the (s, r)-stack with input π^{rc} and output $1, \dots, n$, that is, a sorting history of π^{rc}. The construction is reversible. □

Both maps preserve pattern containment, so they map basis elements to basis elements and preserve lengths. Hence it suffices to prove Theorem 1 for $\mathcal{G}(r, s)$ with $r \le s$: a basis element π of $\mathcal{G}(r, s)$ gives the basis element π^{-1} of $\mathcal{S}(r, s)$, and the basis element π^{rc} of $\mathcal{S}(s, r)$, whose inverse is a basis element of $\mathcal{G}(s, r)$. The sorting duality is due to Atkinson [1, Lemma 1.1], as stated in [2, Proposition 5.5]; we record the explicit capacity-preserving form used below.

2.3 Lazy histories

Lemma 2.4 (lazy normalization). If $\pi \in \mathcal{G}_H(r, s)$, then π is generated by a capacity-H history with the following property. When the next output is already held, it is removed before anything is read. Otherwise the next output is a new maximum v; exactly the unread values up to v are read, and v, the last of them, is removed immediately after its insertion.

Proof. If an insertion of x is immediately followed by the removal of some $y \neq x$, the two can be exchanged: y was held before x arrived, so remove it first, lowering the insertion slot of x by one if y lay above that slot. Both slots only decrease and the intermediate height drops. Moving unnecessary insertions to the right past removals, one output at a time, produces the stated form: every maximal run of insertions is immediately followed by the removal of its last entry, since otherwise that last insertion could be moved further. □

In a lazy history, between two consecutive outputs the stack changes only by inserting the newly read entries and removing the output entry. We use three elementary facts throughout.

- **(D1)** Held entries never change their relative order.
- **(D2)** If a held entry has depth at least $r - 1$, every insertion lands above it.
- **(D3)** An entry can be removed only while its depth is at most $s - 1$.

2.4 Width and capacity

For a nonempty permutation p of length n let

$$w(p) = 1 + \max_{1 \le j \le n} (p_j - j), \qquad w(\varnothing) = 0.$$

The maximum is attained at a left-to-right maximum of p.

Lemma 2.5. Let $p \in \mathcal{G}(r, s)$. Then $p \in \mathcal{G}_H(r, s)$ if and only if $w(p) \le H$.

Proof. Before p_j is output, every value up to $M_j = \max(p_1, \dots, p_j)$ has been read and only $j - 1$ entries have left, so at that moment at least $M_j - j + 1$ entries are held. Taking j at a left-to-right maximum gives height at least $w(p)$. Conversely, Lemma 2.4 applied to any generating history (capacity unbounded) yields a lazy history. In a lazy history the height is largest just after a new maximum p_j has been inserted, where it equals $p_j - j + 1$. So the lazy history has capacity exactly $w(p)$. □

Lemma 2.6. Deleting an entry and standardizing never increases w.

Proof. Let $q = \mathrm{st}(p \setminus v)$, where $v = p_i$. The maximum defining $w(q)$ is attained at a left-to-right maximum $q_{j'}$ of q, coming from an entry $p_j \neq v$. If $j < i$, then $q_{j'} - j' \leq p_j - j$. If $j > i$ and $p_j > v$, then $q_{j'} - j' = (p_j - 1) - (j - 1) = p_j - j$. If $j > i$ and $p_j < v$, let p_k be the maximum of $p_1, \ldots, p_i$, so $k \leq i$ and $p_k \geq v > p_j$. Then $q_{j'} - j' = p_j - (j - 1) \leq (v - 1) - i < p_k - k$. In every case $q_{j'} - j'$ is at most some $p_k - k$. □

3 Unequal widths

Throughout this section $2 \leq r < s$ and $m \geq 0$. Put

$$Q_i = s + 2 + 2i \ (0 \leq i \leq r - 2), \qquad E_j = s + 1 + 2j \ (0 \leq j \leq m + r - 2), \qquad P = r,$$

$$M_0 = s{+}2r{-}2, \quad A_j = M_0{+}2{+}2j \ (0 \leq j < m), \quad M = M_0{+}2m, \quad n = M{+}s{-}r{+}2 = 2s{+}r{+}2m,$$

and define

$$R_{r,s,m} = Q_0, 1,\ Q_1, 2,\ \ldots,\ Q_{r-2}, r-1,\ \ r+1, \ldots, s,\ \ A_0, E_0,\ \ldots,\ A_{m-1}, E_{m-1},$$
$$n,\ P,\ E_m, \ldots, E_{m+r-2},\ \ M+1, \ldots, n-1.$$

The values $1, \ldots, s$ other than $P = r$ appear in the first part, while P follows n; the E_j and the records Q_i, A_j interleave up to M, and $M + 1, \ldots, n$ are the final values, so this is a permutation of $1, \ldots, n$. For $(r, s) = (3, 4)$ the first two members are $6, 1, 8, 2, 4, 11, 3, 5, 7, 9, 10$ and $6, 1, 8, 2, 4, 10, 5, 13, 3, 7, 9, 11, 12$. At $r = 2$ this specializes to the family $\rho_{s,m}$ for $\mathcal{G}(2, s)$, $s \geq 3$, described in the two-push companion note; the identity is also recorded as `family_two` in the unequal-width certificate.

Theorem 3.1. For all $2 \leq r < s$ and $m \geq 0$, $R_{r,s,m}$ is a basis element of $\mathcal{G}(r, s)$, and $w(R_{r,s,m}) = s + 2$.

3.1 Rejection

Consider a lazy history (Lemma 2.4). The first output is the record $Q_0 = s + 2$, so the values $1, \ldots, s + 2$ are read first. Let z be the bottom entry after the first r of them have been inserted; thus $z \leq r$. It has depth $r - 1$, so by (D2) the remaining $s + 1 - r$ nonrecord values are inserted above it, and after Q_0 is output z has depth s.

Suppose $z = t \leq r - 1$. Each low output $1, \ldots, t - 1$ before t removes one entry above z, and each record Q_i in between reads one relay above z (by (D2), since z then has depth $s - 1 \geq r - 1$). So when t is requested, z still has depth s, contradicting (D3). Hence $z = P = r$. After the low outputs, P has $s - 1$ entries above it, and the outputs $r + 1, \ldots, s$ leave exactly $r - 1$ entries above it.

In each step A_j, E_j the record A_j reads one relay and A_j itself, both above P (by (D2)); after A_j and E_j are output, P again has depth $r - 1$. The relay steps read every value up to M, so finally the record n reads the $s - r + 2$ values $M + 1, \ldots, n$, all above P. After n is output, P has depth $(r - 1) + (s - r + 2) - 1 = s$, and P is requested next, contradicting (D3). Thus $R_{r,s,m} \notin \mathcal{G}(r, s)$.

3.2 Every single deletion is generated

Fix a deleted value d. Labels are original, so the input is $1, \ldots, n$ without d. A requested new maximum is always inserted at slot 0 and removed at once, so its transient height counts.

Common rules. The surviving values among $1, \ldots, r$ can first be read in any desired physical order, since at heights below r every slot is available. Afterwards read input only when the next output requires it. While some surviving low value in $1, \ldots, r-1$ is still to be output, insert every other new value at slot $\min(r-1, h)$; the lows placed ahead of that slot stay ahead of the new entries. After all low outputs, if P is held at depth $e \leq r-2$, insert immediately below it, at slot $e+1$; otherwise insert at slot $\min(r-1, h)$. Call P *protected* when its depth is at most $r-2$. Once protected after the low phase, these rules never increase its depth at output boundaries, because a new maximum passes over it only transiently.

Heights. If a surviving original record x occurs at original output position j, the intact lazy history holds $x - j$ entries just after outputting it. In the deletion history this number becomes $x - [d \leq x] - j + [d \text{ was output before } x]$, where $[\cdot]$ is 1 if the condition holds and 0 otherwise; the last bracket is at most the previous one because x is a record. So deletion never raises the height at a surviving record. In the intact word the heights after the records are $r+1$ for the ordinary relay records and $s+1$ for the final record. If an ordinary record is deleted, its relay is already held and is output without reading; the next surviving record satisfies the same bound. Since $s \geq r+1$, every requested relay in an ordinary step is therefore accessible, whatever the relays' order. Before the final output of P at most $s+1$ entries are held; a protected P is accessible, and after its removal at most s entries remain, so every remaining output is accessible.

The cases are as follows.

1. *A low value $t \in \{1, \ldots, r-1\}$.* Arrange the surviving initial lows as P first, then the others in increasing order. They number $r-1$, so the insertions at slot $r-1$ do not disturb the low outputs, and the extra outputs $r+1, \ldots, s$ occur with at most s entries held. The low phase ends with P on top, and the protected rules finish.
2. *An extra value in $r+1, \ldots, s$, or an initial relay $E_0, \ldots, E_{r-2}$.* Put $z = r-1$ and arrange the initial lows as $P, 1, 2, \ldots, r-2, z$. Keep inserting higher values at slot $r-1$. When z is requested the intact height would be $s+1$; the deletion lowers it to s, so z is at depth $s-1$ and accessible. Afterwards P is still first. A missing relay just leaves one fewer held relay until its omitted output is passed.
3. *A record Q_i.* Put $z = i+1$ and arrange the lows as P, the other lows increasing, then z last. If $i > 0$, just before the missing record would be read the height is s with z last; output z from depth $s-1$ without reading. If $i = 0$, output $z = 1$ from depth $r-1 < s$ right after the prefill. The postponed relay is read with the next record, below P and the remaining low prefix, so P ends first and protected.
4. *The value P.* Arrange the surviving lows increasing. After the low and extra outputs, each relay step holds at most r entries after its record is output, and the final step at most s, so every requested output is accessible.
5. *A relay E_j with $j \geq r-1$.* Use the unmodified initial history; P reaches depth $r-1$. At the step that would introduce E_j, only the record is read, and the requested older relay E_{j-r+1} is held; removing it leaves $r-1$ entries and protects P at depth $r-2$. The rules finish, also when the skipped output of E_j lies in the final segment.
6. *An ordinary record A_j.* Run the unmodified history up to step j. The relay E_j is held; output it before reading. Again $r-1$ entries remain and P is protected; the postponed relay is read with the next record.
7. *The final record n.* After the relay steps P has depth $r-1 < s$; output it, then the held relays. The values $M+1, \ldots, n-1$ are then successive new maxima.

8. *A final value in* $M+1,\dots,n-1$. The intact history leaves P last among r held entries. The final record reads one entry fewer, so after it is output the height is s with P at depth $s-1$. Output P; every remaining entry is then accessible.

These cases exhaust the low values, extra values, relays, records and final values, which proves the first claim of Theorem 3.1 together with Section 3.1 and Lemma 2.1.

3.3 Width

Just after the records Q_i, A_j and n are inserted, the lazy history holds $s+2$, $r+2$ and $s+2$ entries respectively, so $w(R_{r,s,m}) = s+2$. □

4 Equal widths

Throughout this section $r \geq 3$, and the machine is the (r,r)-stack. The unequal construction cannot work here: an entry at depth $r-2$ can be undercut, so the bottom entry is no longer pinned.

4.1 The families

Name the values

$$Q_i = r+2i \ (1 \leq i \leq r-1), \quad E_j = r+1+2j \ (0 \leq j \leq r-2), \quad P = r,$$

$$C = 3r-1, \quad U = 3r, \quad M_1 = 3r+1, \quad V = 3r+2, \quad M_2 = 3r+3,$$

and the prefix

$$\Pi_r = Q_1, 1,\ Q_2, 2,\ \dots,\ Q_{r-1}, r-1,\ E_0,\ M_1,\ P,\ M_2,\ U,\ E_1.$$

The entry C is the *carrier*. For $j \geq 0$, step j uses $a_j = 3r+4+3j$, $b_j = a_j+1$ and the record $R_j = a_j+2$. A queue of $r-2$ old entries starts as $O_0 = (E_2, E_3, \dots, E_{r-2}, V)$. Step j outputs R_j, then a_j, then the head (oldest entry) of the queue; the head leaves the queue and b_j joins it at the back. Let O_k be the queue after k steps, and $a = a_k$. Then

$$A_r(k) = \Pi_r,\ \text{step } 0, \dots, \text{step } k-1,\ a+3,\ a,\ C,\ \text{sort}(O_k),\ a+1,\ a+2,$$

$$B_r(k) = \Pi_r,\ \text{step } 0, \dots, \text{step } k-1,\ a+2,\ a,\ a+4,\ C,\ \text{sort}(O_k),\ a+1,\ a+3,$$

where sort lists in increasing order. These are permutations of lengths $3r+7+3k$ and $3r+8+3k$: Π_r together with $O_0 \cup \{C\}$ is $\{1, \dots, 3r+3\}$, each step uses the next three values, and the final segment uses the last four or five together with C and O_k. For example

$$A_3(0) = 5,1,7,2,4,10,3,12,9,6,16,13,8,11,14,15,$$
$$B_4(0) = 6,1,8,2,10,3,5,13,4,15,12,7,18,16,20,11,9,14,17,19.$$

Theorem 4.1. For all $r \geq 3$ and $k \geq 0$, $A_r(k)$ and $B_r(k)$ are basis elements of $\mathcal{G}(r,r)$. Moreover $w(B_r(k)) = r+2$ and $w(A_r(k)) = r+3$.

4.2 Rejection

All histories are lazy (Lemma 2.4).

Lemma 4.2 (prefix). In every lazy history whose outputs begin with Π_r, afterwards C is the bottom entry, exactly the entries of O_0 lie above it, and the next unread value is a_0.

Proof. The record Q_1 reads $1, \ldots, r+1$. Among the first r of them let z be the bottom entry; E_0 is inserted at slot at most $r-1$ into a stack of height r, so z stays at the bottom with depth r. For $i = 1, \ldots, r-1$ in turn: before output i, z has depth r, so $z \neq i$ by (D3); the output leaves z at depth $r-1$; and for $i \leq r-2$ the record Q_{i+1} reads E_i, which lands above z by (D2), restoring depth r. Hence $z = P$, with $E_0, \ldots, E_{r-2}$ above it. Outputting E_0 leaves P at depth $r-2$.

The record M_1 reads C and then U. If C were inserted above P, then P would have depth $r-1$, so U would land above it by (D2), and P, requested right after M_1, would be at depth r, contradicting (D3). So C is inserted below P and becomes the bottom entry, at depth $r-1$; then U lands above it. Outputting P leaves C at depth $r-1$ under $E_1, \ldots, E_{r-2}, U$. The record M_2 reads V, which lands above C; outputting U and E_1 leaves exactly O_0 above C, at depth $r-2$. □

Lemma 4.3 (step). Suppose C is the bottom entry at depth $r-2$ under the current queue, and the next unread value is a_j. Then step j leaves C at the bottom at depth $r-2$ under the new queue.

Proof. The record R_j reads a_j and b_j. If a_j were inserted below C, it would be the bottom entry at depth $r-1$, so b_j would land above it and a_j, requested right after R_j, would be at depth r. So a_j lies above C, which then has depth $r-1$, and b_j lands above C as well. The outputs a_j and the queue head, both above C, leave the new queue above it. □

Lemma 4.4 (finals). In the state of Lemma 4.3 after k steps, the final segments of $A_r(k)$ and $B_r(k)$ cannot be output.

Proof. For $A_r(k)$ the record $a+3$ reads $a, a+1, a+2$. As in Lemma 4.3, a lies above C, and then $a+1, a+2$ land above C. After a is output, the r entries of $O_k \cup \{a+1, a+2\}$ lie above C, which is requested next at depth r. For $B_r(k)$ the record $a+2$ reads $a, a+1$; again both lie above C, and after a is output C has depth $r-1$. The record $a+4$ reads $a+3$, which lands above C, and C is requested at depth r. □

Lemmas 4.2–4.4 and induction on k show that neither $A_r(k)$ nor $B_r(k)$ lies in $\mathcal{G}(r, r)$, with no capacity bound.

4.3 Every single deletion is generated

Histories are described by where each newly read entry is inserted: $x@j$ inserts x at slot j, while $x^{\uparrow y}$ and $x_{\downarrow y}$ insert x directly above or below y. Every record is inserted at slot 0 and removed at once. Each claim states the slot or depth to be checked; all slots are at most $\min(r-1, h)$ and all removals are at depth at most $r-1$.

The standard prefix. With no deletion in Π_r:

1. Q_1: insert $1, \ldots, r-1$, then P at the bottom and $E_0^{\uparrow P}$, giving $(1, \ldots, r-1, E_0, P)$; output Q_1.
2. For $i = 1, \ldots, r-1$: output i from the top; if $i \leq r-2$, the record Q_{i+1} reads $E_i^{\uparrow P}$, at slot $r-1$.
3. Output E_0, leaving $(E_1, \ldots, E_{r-2}, P)$.
4. M_1: insert $C_{\downarrow P}$ at slot $r-1$, then $U^{\uparrow C}$ at slot $r-1$; output M_1, then P at depth $r-2$.
5. M_2: insert $V^{\uparrow C}$ at slot $r-1$; output M_2, then U at depth $r-2$, then E_1.

The result is **regime S**: the stack $(E_2, \ldots, E_{r-2}, V, C)$, as forced by Lemma 4.2. Two further regimes occur after a deletion.

- **Regime L:** C is held at depth at most $r-3$; other old entries may lie above or below it in any order, and some may still be unread. Unread old entries are read first at the next record, in increasing order, each inserted just below C.
- **Regime Z:** C has been deleted.

In every history of this section, each entry read so far other than C and the current queue entries has already been output.

Steps. In regime S a step inserts $a_j@(r-2)$ just above C and $b_j@(r-1)$ between a_j and C, then outputs R_j, a_j from depth $r-2$, and the head from the top; regime S persists. In regime L, with C at depth $e \le r-3$, a step reads any pending old entries just below C, inserts a_j just below C at slot $e+1$ and b_j just below a_j at slot $e+2 \le r-1$, and outputs R_j and a_j from depth $e+1$. At that moment the stack holds only C, at most $r-2$ queue entries and b_j, at most r entries in all, so the head is accessible. Nothing is inserted above C, so regime L persists. A step in regime Z is the same with C omitted, inserting $a_j@0$ and $b_j@1$.

Deletions inside a step j (regime S on arrival). Each case ends in regime L with C at depth $r-3$.

- R_j: a_j is now a new maximum with nothing to read; output it, then the head from the top. The value b_j stays unread.
- a_j: insert $b_j@(r-1)$ just below C; output R_j, then the head.
- b_j: insert $a_j@(r-1)$ just below C; output R_j, then a_j from depth $r-1$, then the head.

Deletions inside Π_r. Several cases pass through the state

$$X = (P, E_2, \ldots, E_{r-2}, E_1) \quad \text{before } M_1, \text{ with } C, U, V, M_1, M_2 \text{ unread},$$

from which the *tail* is: M_1 reads $C@1$ and $U@(r-1)$, giving $(P, C, E_2, \ldots, E_{r-2}, U, E_1)$; output P; M_2 reads $V@(r-1)$; output U at depth $r-2$ and E_1 at depth $r-1$. The result is regime L with C on top.

- **(P1) Q_1.** Output 1 as a new maximum. Q_2 reads $2, \ldots, r-1$ at slots $0, \ldots, r-3$, then $P@(r-2)$, $E_0@(r-1)$ and $E_1@(r-1)$, giving $(2, \ldots, r-1, P, E_1, E_0)$. Output 2; for $t \ge 3$, Q_t reads $E_{t-1}^{\uparrow E_1}$, at slot at most $r-2$, and t is output. Output E_0 from depth $r-1$, reaching X; run the tail.
- **(P2) A low value i, E_0, or Q_t with $2 \le t \le r-1$.** The record Q_1 arranges its entries as (lows except i, P, E_0), each inserted at the bottom, when i is deleted; as $(1, \ldots, r-1, P)$ when E_0 is deleted; and as $(1, \ldots, t-1, t+1, \ldots, r-1, P, E_0, t)$ when Q_t is deleted, inserting $1, \ldots, t-1$ and t at the bottom and the later entries above t (slots $x-2 \le r-3$ for a low x, $r-2$ for P, $r-1$ for E_0). Then the lows are output in order from the top, and each surviving Q_{x+1} reads its pending relays: E_1 goes above E_0, or below P if E_0 is deleted, and every later E_x goes above E_1, at slot at most $r-2$. If Q_t is deleted, t is output right after $t-1$, from depth exactly $r-1$, and its two relays are read with Q_{t+1}; when $t = r-1$, E_{r-2} is read with M_1, at the bottom if it equals E_1 (that is, $r=3$) and above E_1 otherwise. Finally E_0 is output from depth at most $r-1$, reaching X; run the tail.

- **(P3)** E_1**.** Run the standard steps 1–3, giving $(E_2, \ldots, E_{r-2}, P)$. M_1 reads $C^{\uparrow P}$ and U at the bottom; output P at depth $r-2$. M_2 reads V at the bottom; output U at depth $r-2$. The result $(E_2, \ldots, E_{r-2}, C, V)$ is regime L.
- **(P4)** U**.** Run steps 1–3, giving $(E_1, \ldots, E_{r-2}, P)$. M_1 reads $C_{\downarrow P}$; output P. M_2 reads V at the bottom; output E_1. The result $(E_2, \ldots, E_{r-2}, C, V)$ is regime L.
- **(P5)** M_1**.** Run the low phase with $E_1^{\uparrow P}$ and every later $E_x^{\uparrow E_1}$ (slot at most $r-2$); before E_0 is output the stack is $(E_0, E_2, \ldots, E_{r-2}, E_1, P)$. Output E_0, then P at depth $r-2$. M_2 reads C@0, U@1 and V@$(r-1)$, giving $(C, U, E_2, \ldots, E_{r-2}, V, E_1)$. Output U at depth 1 and E_1 at depth $r-1$. The result is regime L.
- **(P6)** P**.** As (P5), except that E_0 and E_1 go to the bottom, at slot $r-1$, and M_1 reads C@0 and U@1; M_2 reads V@$(r-1)$; output U, then E_1. The result is regime L.
- **(P7)** M_2**.** Run steps 1–3. M_1 reads $C_{\downarrow P}$ and $U_{\downarrow P}$; output P at depth $r-2$, then U at depth $r-2$, then E_1. The result $(E_2, \ldots, E_{r-2}, C)$, with V unread, is regime L once V is read at the bottom.
- **An entry of** O_0 (E_x with $x \geq 2$, or V): the standard prefix with that entry omitted leaves C at depth $r-3$, which is regime L.
- C**:** the standard prefix with $U_{\downarrow P}$ in step 4 and V at the bottom in step 5 leaves $(E_2, \ldots, E_{r-2}, V)$, which is regime Z.

The final segment from regime L or Z. Let C have depth e in regime L. First read any pending old entries just below C. For $A_r(k)$, insert a, $a+1$ and $a+2$ at slots $e+1$, $e+2$, $e+2$; output $a+3$, then a from depth $e+1$, then C from depth e. For $B_r(k)$, insert a@$(e+1)$ and $(a+1)$@$(e+2)$; output $a+2$ and a; insert $a+3$ just below C; output $a+4$, then C from depth e. In regime Z skip C and insert at the top instead (slots $0, 1, 1$ for A; $0, 1$ and then 0 for B). In every case at most r entries remain afterwards, so they can be output in the required order.

The final segment from regime S. Here the stack is (queue, C), C has depth $r-2$, and slot $r-1$ lies just below C. In each row every requested entry is within depth $r-1$, and at most r entries remain after C.

Family	Deleted	History
A	$a+3$	output a; output C; output O_k; output $a+1$; output $a+2$
A	a	insert $a+1$ and $a+2$ below C; output $a+3$; output C, then the rest
A	$a+1$ or $a+2$	insert a@$(r-2)$ and the other at slot $r-1$; output $a+3$, a (depth $r-2$), C (depth $r-1$), then the rest
B	$a+2$	output a; insert $a+1$ and $a+3$ below C; output $a+4$; output C, then the rest
B	a	insert $a+1$ below C; output $a+2$; insert $a+3$ below C; output $a+4$; output C, then the rest

Family	Deleted	History
B	$a+4$	insert a@$(r-2)$ and $(a+1)$@$(r-1)$; output $a+2$, a, C (depth $r-1$), O_k, $a+1$; output $a+3$
B	$a+1$	insert a@$(r-1)$ below C; output $a+2$, then a; insert $a+3$ below C; output $a+4$; output C, then the rest
B	$a+3$	insert a@$(r-2)$ and $(a+1)$@$(r-1)$; output $a+2$, then a; output $a+4$; output C (depth $r-1$), then the rest

Coverage. The values of $A_r(k)$ split as follows: the $2r+4$ values of Π_r ((P1)–(P7)); C (regime Z); the $r-2$ entries of O_0 (regime L); three values in each step $j<k$; and the four final values $a,\dots,a+3$. A queue entry other than those of O_0 is some b_j, counted in its step. For $B_r(k)$ the last group has the five values $a,\dots,a+4$. After a switch to regime L or Z, every later step and the final segment use that regime's rules. This proves the first claim of Theorem 4.1.

4.4 Width

Every record of $B_r(k)$ (the Q_i, M_1, M_2, each R_j, and $a+2$, $a+4$) exceeds its position by exactly $r+1$, and every other entry by less; so $w(B_r(k))=r+2$. In $A_r(k)$ the record $a+3$ exceeds its position by $r+2$, so $w(A_r(k))=r+3$. □

5 Capacity thresholds

Proof of Theorem 3(a). First suppose $r\le s$, so $H\le s+1$. Let p be a permutation with $w(p)\le H$. Generate it lazily, inserting every newly read entry at slot 0. A requested new maximum is then on top. An already held output is requested from a stack of height at most $H-1\le s$, so it has depth at most $s-1$ and is accessible. The transient height at a record is at most $w(p)\le H$. Conversely every history generating p has height at least $w(p)$ (Lemma 2.5). Hence $\mathcal{G}_H(r,s)=\{p : w(p)\le H\}$, and by Lemma 2.2, $\pi\in\mathcal{S}_H(r,s)$ if and only if $w(\pi^{-1})\le H$. For $p=\pi^{-1}$ one has $w(p)=1+\max_i(i-\pi_i)=1+D(\pi)$ when π is nonempty, since $p_j-j=i-\pi_i$ for $j=\pi_i$.

For $r>s$, Lemma 2.3 gives $\pi\in\mathcal{S}_H(r,s)$ if and only if $\pi^{rc}\in\mathcal{G}_H(s,r)$. Apply the generated-side characterization just proved for $s<r$ and use $w(\pi^{rc})=1+D(\pi)$ for nonempty π. This proves the same sortable-side characterization; the generated side follows by inversion. The empty permutation belongs to both classes at every capacity.

For the basis, call an entry's *excess* the number of larger entries before it. The maximum drop of π equals its largest excess: the drop at an entry is its excess minus the number of smaller entries after it, and a suffix minimum at or after an entry of largest excess keeps all those larger predecessors and has no smaller successor. Excess can only decrease when entries are deleted. Thus $D(\pi)\ge H$ exactly when π contains a pattern of length $H+1$ ending in its minimum, all such patterns are minimal, and there are $H!$ of them for $H\ge 1$. For $H=0$ the class contains only the empty permutation. □

Proof of Theorem 3(b). Suppose first $r < s$, so $H \geq s+2$. By Theorem 3.1, $R_{r,s,m} \notin \mathcal{G}(r,s)$, hence not in $\mathcal{G}_H(r,s)$, while each single deletion q lies in $\mathcal{G}(r,s)$ and has $w(q) \leq w(R_{r,s,m}) = s+2 \leq H$ by Lemma 2.6, hence lies in $\mathcal{G}_H(r,s)$ by Lemma 2.5. So each $R_{r,s,m}$ is a basis element of $\mathcal{G}_H(r,s)$, and these have unbounded lengths. When $r = s \geq 3$ the same argument applies to $B_r(k)$ with $w(B_r(k)) = r+2$. Lemmas 2.2 and 2.3 preserve capacity, which gives $\mathcal{S}_H(r,s)$ and the case $r > s$. □

Proof of Theorem 1 and Corollary 2. The families of Theorems 3.1 and 4.1 have unbounded lengths, so they are infinite sets of basis elements of $\mathcal{G}(r,s)$ for $r \leq s$, $(r,s) \neq (2,2)$; Lemmas 2.2 and 2.3 transfer them to $\mathcal{S}(r,s)$ and to $r > s$. The corollary combines this with Atkinson's finite bases for $\min(r,s) = 1$ and $r = s = 2$ [1]. □

Following [6], a $\mathcal{C}$-machine generating from the input $1, \ldots, n$ may at each step *bypass* (send the next input entry directly to the output), *push* (insert the next input entry anywhere in the container, provided the contents remain order-isomorphic to a member of $\mathcal{C}$), or *pop*; in a $(\mathcal{C}, s)$-machine [2] a pop removes any one of the leftmost s entries of the container.

Proposition 5.1. For all $r, s \geq 1$, the $(\mathcal{C}_r, s)$-machine generates exactly the class $\mathcal{G}(r,s)$.

Proof. Identify the container, read from left to right, with the stack read from the top. In generating mode every pushed entry exceeds all stored entries. Consider a stack history generating π. An entry x inserted at slot $j \leq r-1$ has j entries before it, all smaller. Entries inserted later are larger than x, and removals only delete entries, so throughout the history fewer than r smaller entries precede x. Hence the contents always avoid $S_r \oplus 1$, every insertion is a legal push of the $(\mathcal{C}_r, s)$-machine, and every removal at depth at most $s-1$ is a legal pop. Conversely, in the machine a push of the new maximum into position j (with j entries before it) keeps the contents in $\mathcal{C}_r$ only if $j \leq r-1$, because any r of the j smaller entries before it, followed by it, form a pattern in $S_r \oplus 1$; so it is a legal stack insertion. A pop is a legal stack removal, and a bypass is an insertion at slot 0 followed at once by the removal of that entry. The first half of the argument does not use bypass, so the conclusion also holds for the machine without bypass. □

Proof of Corollary 4. The class $\mathcal{C}_r$ is defined by avoiding the finite set $S_r \oplus 1$, and $S_2 \oplus 1 = \{123, 213\}$. By Proposition 5.1 the class generated by the $(\mathcal{C}_r, s)$-machine is $\mathcal{G}(r,s)$, whose basis is infinite by Theorem 1. □

As a finite check, a brute-force comparison of the permutations generated by the stack and by the machine, with and without bypass, agreed for ten width pairs with $r, s \leq 4$ and all lengths up to 7 or 8.

6 Formal verification

The infinite-basis statements and the precisely scoped capacity-threshold statements below are checked by the Lean 4 kernel (version 4.34.1, core library only, no mathlib) in three certificates that build on one another. The machine is formalized directly: a step either inserts an input entry into a list at a position below the push width, or removes the entry at a position below the pop width; a run is a sequence of steps recording its exact input and output words; and this relational semantics is proved equivalent to an executable slot-indexed interpreter. Basis elements are formalized as permutations of $\{0, \ldots, n-1\}$ that no run generates from increasing input, and all of whose single deletions, with original labels kept, some run generates. Capacity is formalized as a run in which every visited stack, before and after each step, has length at most H.

The certificates prove, with only the standard axioms (propositional extensionality, choice and quotient soundness):

- for all $2 \le r < s$ and m, that the family of Section 3 consists of basis elements of $\mathcal{G}(r, s)$, and its capacity version for $H \ge s + 2$;
- inversion and reverse-complement duality, with and without capacity, hence Theorem 1 for unequal widths in both classes;
- the finite side of Theorem 3, that is, the characterization of $\mathcal{G}_H(r, s)$ and $\mathcal{S}_H(r, s)$ for $H \le \max(r, s) + 1$ and the fact that every capacity-H basis element then has length $H + 1$, for all positive widths;
- for all $r \ge 3$ and k, that $A_r(k)$ and $B_r(k)$ are basis elements of $\mathcal{G}(r, r)$, their inverses of $\mathcal{S}(r, r)$, and $w(B_r(k)) \le r + 2$;
- the combined statements: Theorem 1 for all $r, s \ge 2$ except $(2, 2)$, and, for the same widths, basis length exactly $H + 1$ below the capacity threshold and basis elements beyond every length bound above it. A separate finite-cardinality certificate identifies the entire generated and sortable bases over all lengths and proves that each has $H!$ elements whenever $r, s > 0$ and $H \le \max(r, s) + 1$, including $H = 0$.

A fourth certificate, `formal/container`, formalizes Proposition 5.1 and Corollary 4. Its three new modules reuse the preceding 30 core/Std modules unchanged. They prove literal pattern avoidance of $S_r \oplus 1$, the positive-width generation equivalence for every strictly increasing input with or without bypass, and transfer of the original-machine infinite basis to the container machine. The frozen 33-module closure is supplied for source-only checking with Lean 4.34.1. The recorded axiom audit found only the standard foundational axioms listed above. This is an **unbounded-capacity** equivalence: simulating a bypass uses a real intermediate held token, so the certificate does not assert preservation of a fixed physical capacity. The exact source hashes and scope are recorded in the source distribution.

The deletion histories in Lean are literal histories that follow Sections 3.2 and 4.3, sometimes in a different but equivalent order. The finite bases of Atkinson [1] used in Corollary 2 are not formalized. The added obstruction-width certificate proves the exact displacement identities $w(R_{r,s,m}) = s + 2$, $w(A_r(k)) = r + 3$ and $w(B_r(k)) = r + 2$, for $2 \le r < s$, $m \ge 0$, and $r \ge 3$, $k \ge 0$, respectively. These identities describe displacement widths and do not assert successful runs of the rejected families. The source-only cardinality and width packets, with their exact scopes and reproduction commands, are in `formal/basis-cardinality-width/`. The correspondence between the formal definitions of the families and the displayed ones was checked by evaluation on small members and by direct comparison with the written definitions.

As further checks, the written deletion histories were replayed by a literal machine simulator: 29,562 histories for the 819 members of the unequal family with $2 \le r \le 10$, $r < s \le 13$ and $m \le 12$, and 54,600 histories for the equal-width members with $3 \le r \le 30$ and $k \le 12$. A pruned exhaustive search confirmed rejection and minimality for 133 of 136 equal-width members with $r \le 8$; the other three runs were stopped without a conclusion. These finite checks are not part of the proof.

7 Remarks

Literature. Within recorded bounds we searched the works citing [1] in several citation indexes, recent arXiv work (including 2026 papers by authors active on sorting machines that were located in the recorded search), the OEIS, and the literature on related machines. We found no earlier proof of Theorem 1, Theorem 3 or Corollary 4. Version 2 of Vatter's list [2] (February 2026) still records the finiteness question as open, and Bóna's survey [7] records finite bases only for $r = 1$, $s = 1$ and $r = s = 2$. Some sources could not be read in full, including Atkinson's survey [4]; absence

from a bounded search is weak evidence, and we make no priority claim. Related machines behave differently. The fork-stacks of Albert and Atkinson, which move blocks of entries, have finitely many minimal unsortable permutations for all widths [8]. The closest analogue of Theorem 3 that we know is the result of Elder and of Elder and Goh that a stack of depth t followed by an infinite stack is finitely based exactly when $t \leq 2$ [9, 10]. Explicit infinite families of minimal unsortable permutations go back to Pratt's work on deques and parallel stacks [11]; see also Atkinson, Murphy and Ruškuc [12] on infinite antichains in classes of sortable permutations.

The bases themselves. Theorem 1 does not determine the bases. An exhaustive computation for the $(3, 3)$-stack, stopped at length 13, finds 1764 sortable-input basis elements of lengths 7 to 13 (finite data). Our displayed families begin at length 16; that is not a claim that other basis elements stop at length 13. The families were selected for proof, not to be exhaustive.

Enumeration. Vatter [2] also asks about the enumeration of these classes. An infinite basis does not by itself rule out an algebraic generating function, and we do not address enumeration here.

References

[1] M. D. Atkinson, Generalized stack permutations, *Combinatorics, Probability and Computing* 7 (1998), no. 3, 239–246. doi:10.1017/S096354839800354X.

[2] V. Vatter, An assortment of problems in permutation patterns: unimodality, equivalence, derangements, and sorting, arXiv:2602.16355 (2026).

[3] D. E. Knuth, *The Art of Computer Programming*, Vol. 1: Fundamental Algorithms, Addison-Wesley, 1968, Section 2.2.1.

[4] M. D. Atkinson, Permuting machines and permutation patterns, in *Permutation Patterns*, London Math. Soc. Lecture Note Ser. 376, Cambridge University Press, 2010, 67–88. doi:10.1017/CBO9780511902499.004.

[5] L. de Moura and S. Ullrich, The Lean 4 theorem prover and programming language, in *Automated Deduction – CADE 28*, Lecture Notes in Comput. Sci. 12699, Springer, 2021, 625–635. doi:10.1007/978-3-030-79876-5_37.

[6] M. H. Albert, C. Homberger, J. Pantone, N. Shar and V. Vatter, Generating permutations with restricted containers, *J. Combin. Theory Ser. A* 157 (2018), 205–232. doi:10.1016/j.jcta.2018.02.006.

[7] M. Bóna, A survey of stack-sorting disciplines, *Electron. J. Combin.* 9(2) (2003), #A1. doi:10.37236/1693.

[8] M. H. Albert and M. D. Atkinson, Sorting with a forklift, *Electron. J. Combin.* 9(2) (2003), #R9. doi:10.37236/1681.

[9] M. Elder, Permutations generated by a stack of depth 2 and an infinite stack in series, *Electron. J. Combin.* 13 (2006), #R68. doi:10.37236/1094.

[10] M. Elder and Y. K. Goh, Permutations sorted by a finite and an infinite stack in series, in *Language and Automata Theory and Applications (LATA 2018)*, Lecture Notes in Comput. Sci., Springer, 2018, 220–231. doi:10.1007/978-3-319-77313-1_17.

[11] V. R. Pratt, Computing permutations with double-ended queues, parallel stacks and parallel queues, in *Proceedings of the Fifth Annual ACM Symposium on Theory of Computing*, 1973, 268–277. doi:10.1145/800125.804058.

[12] M. D. Atkinson, M. M. Murphy and N. Ruškuc, Partially well-ordered closed sets of permutations, *Order* 19 (2002), 101–113. doi:10.1023/A:1016500300436.